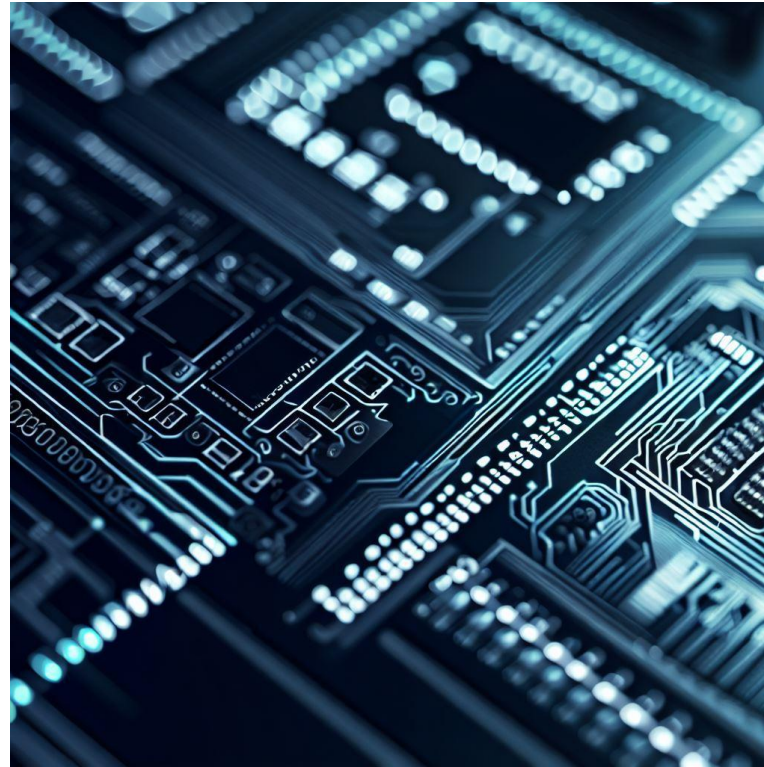
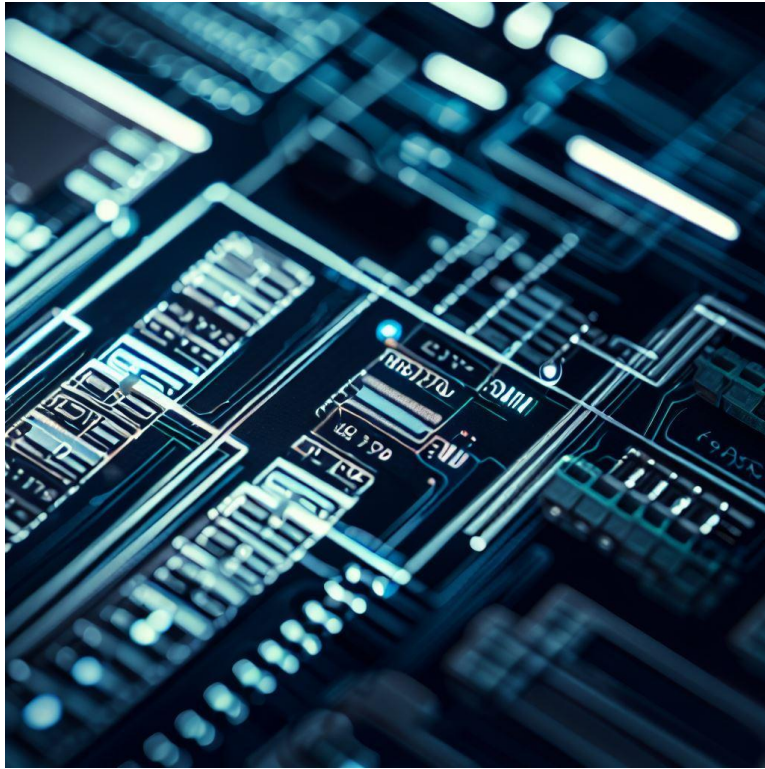




THE PROGRAMMABLE INPUT/OUTPUT

State machines – PIO Programs – Control Flow – Registers – Stalling – Pin Mapping
– IRQ Flags – Interactions Between State Machines – Instruction Set – Functional
Details

THE PROGRAMMABLE INPUT/OUTPUT

PIO	Outline Programmable Input Output	U
PIO Programs, Control Flow	Illustrate the PIO instructions and the control flow	Ap
Registers	Show the need for different registers in PIO	Ap
Stalling, Pin Mapping	Generalize stalling and pin mapping	U
IRQ Flags, Interactions Between State Machines	Summarize IRQ flags and interaction between state machines	U
Instruction Set	Illustrate the instruction set with example	U
Functional Details	Describe the functional details of state machines	R

What is **PIO**?

- Programmable Input and Output
- State machine (tiny processors)
- 2 PIOs and each PIO has 4 state machines



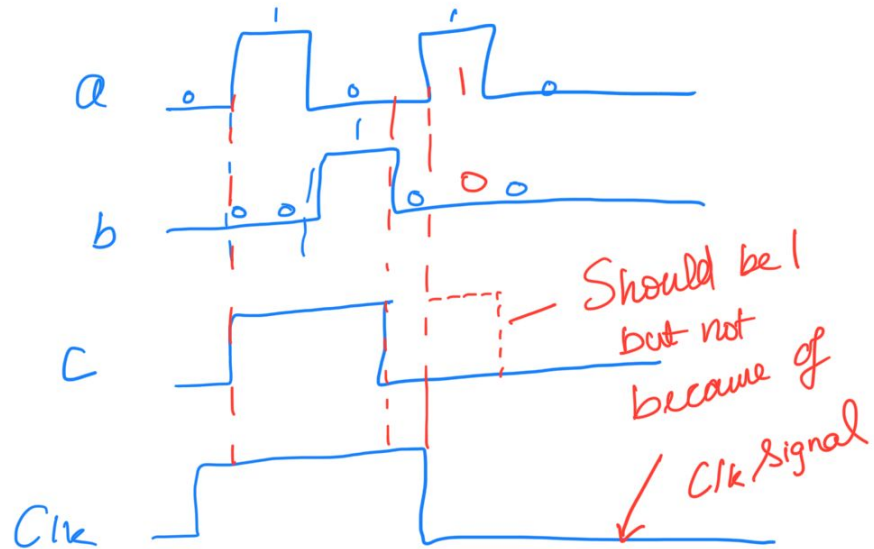
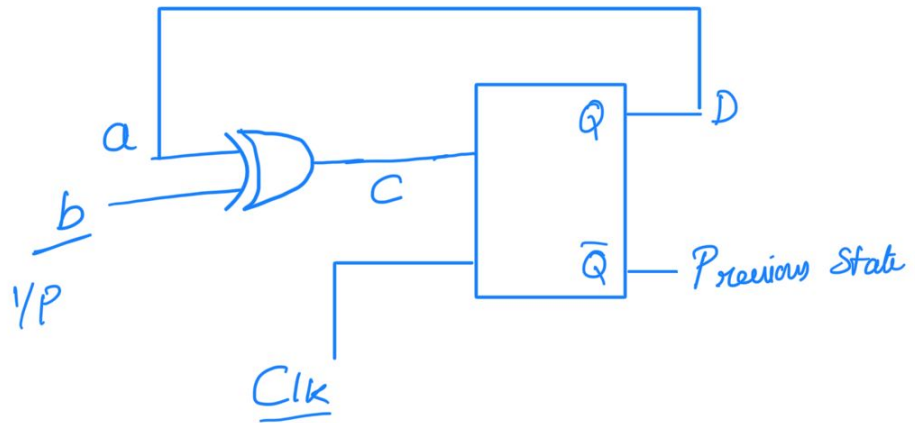


Moore-Machine »output depends only on the current state

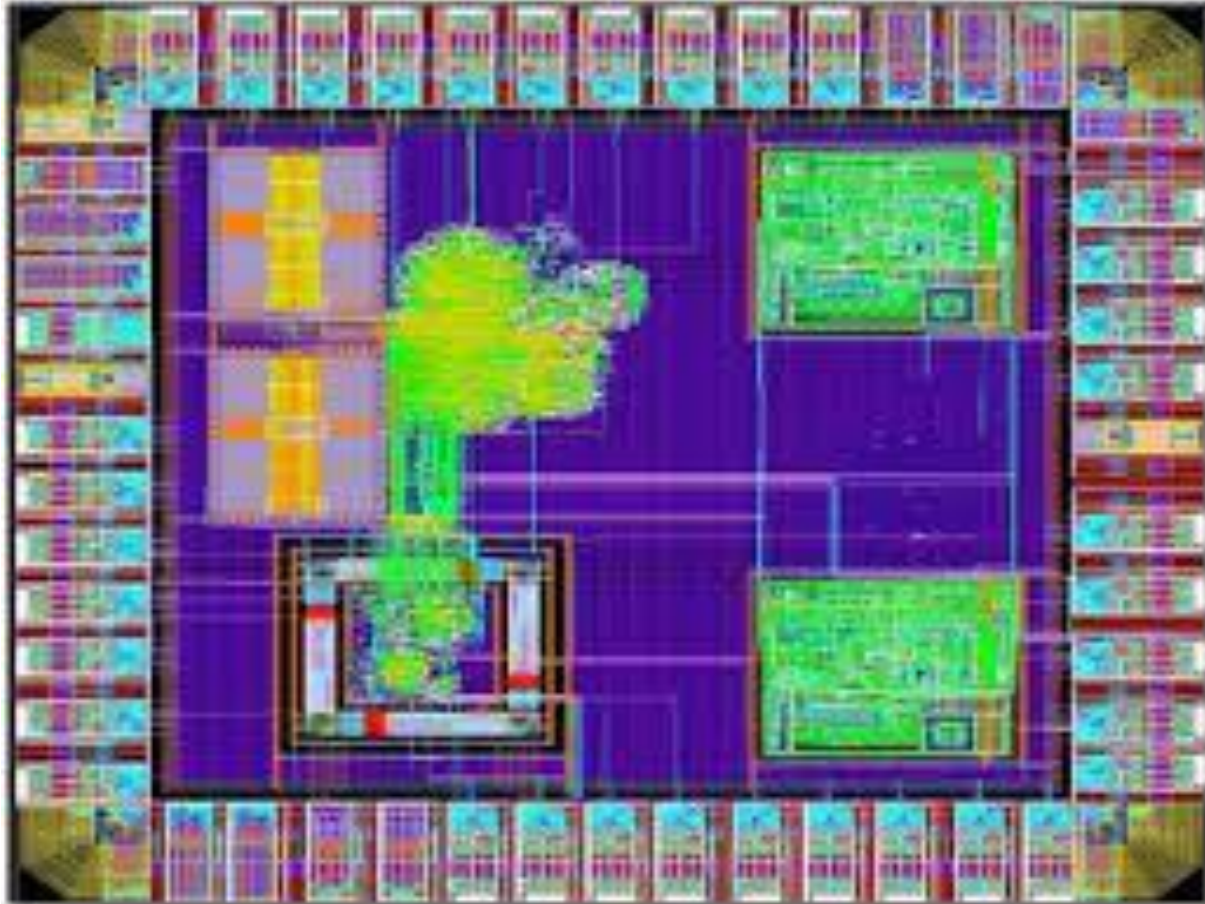
Now, let's simplify this



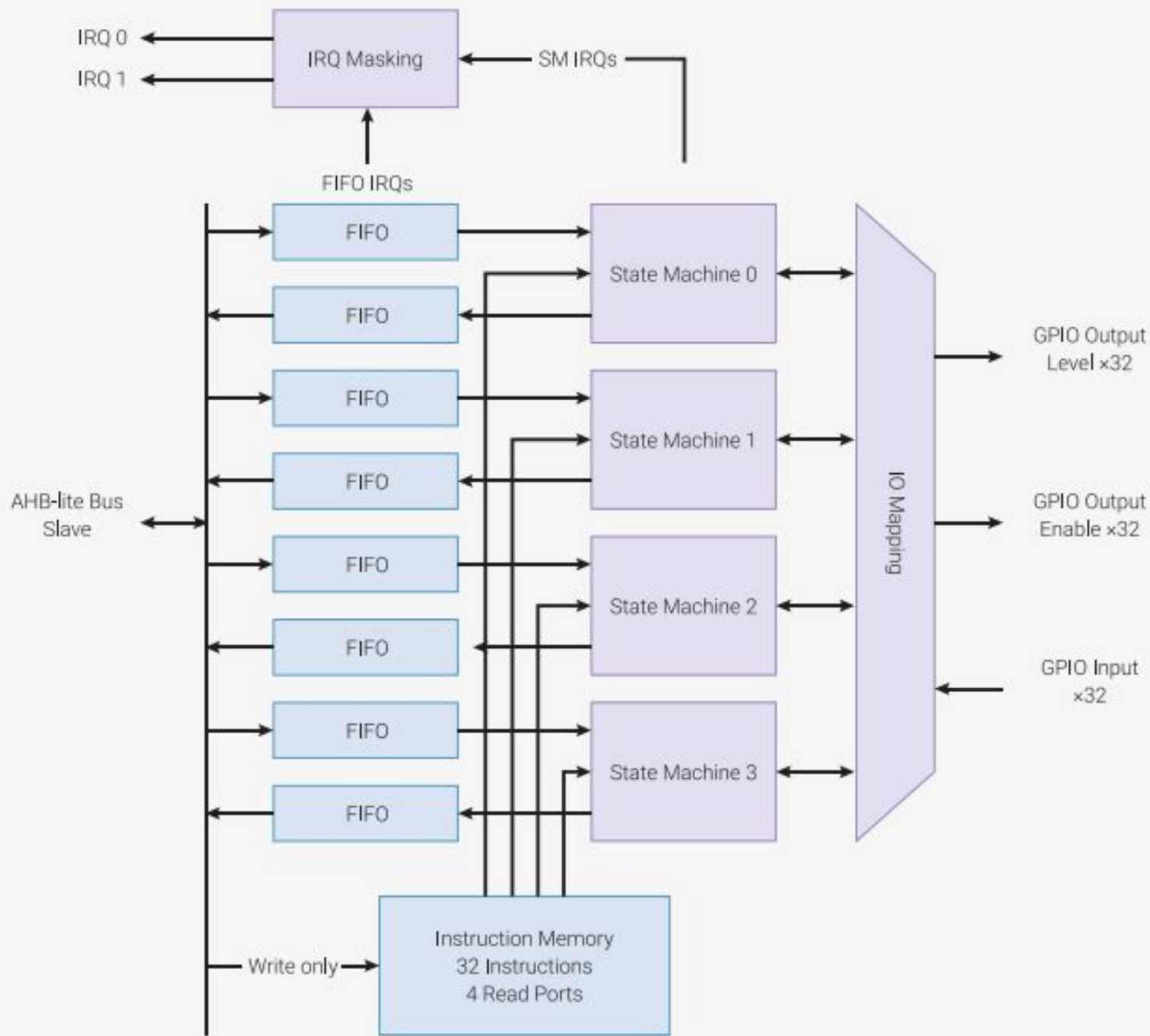
Now, let's simplify this



Tiny yet mighty PIOs



- The silicon region occupied by the PIO are same as I2C and SPI, however they can transform to perform different operation



PIO x2

ILO

Outline Programmable
Input Output

State Machine

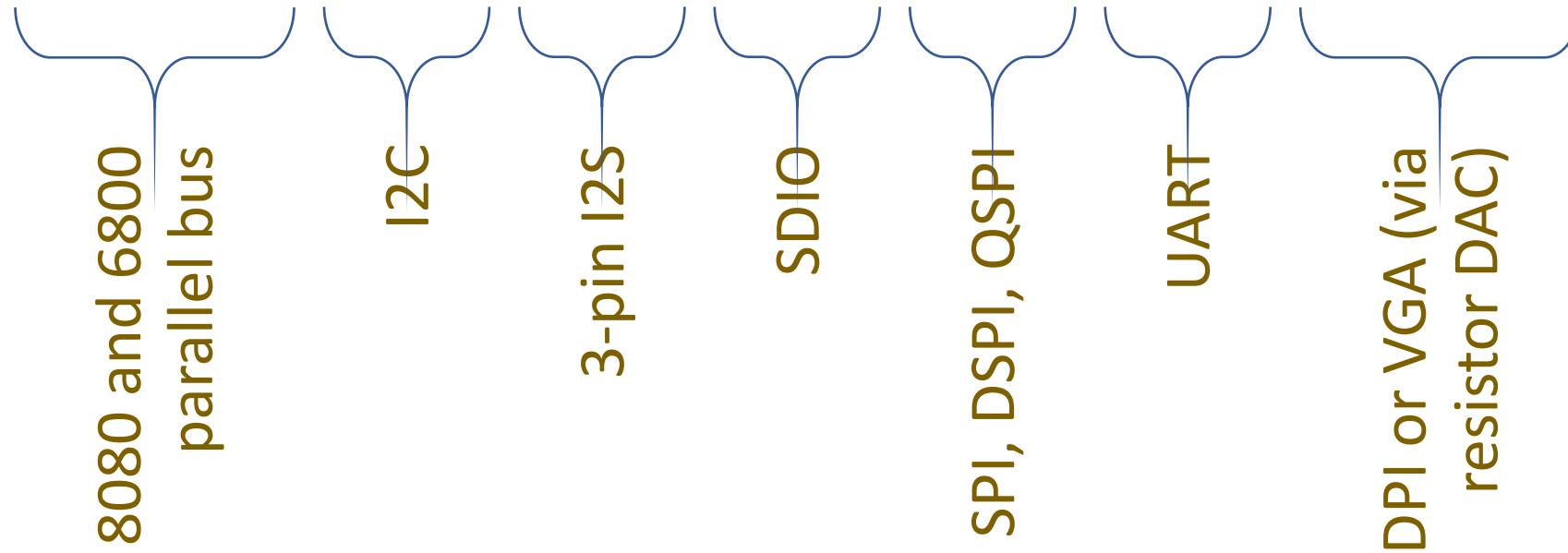
PIO is programmable in the same sense as a processor.

Each state machine is equipped with:

- **scratch registers**
- **shift registers**
- **bus FIFO**
- **clock divider**
- **Flexible GPIO**
- **DMA**
- **IRQ flag**

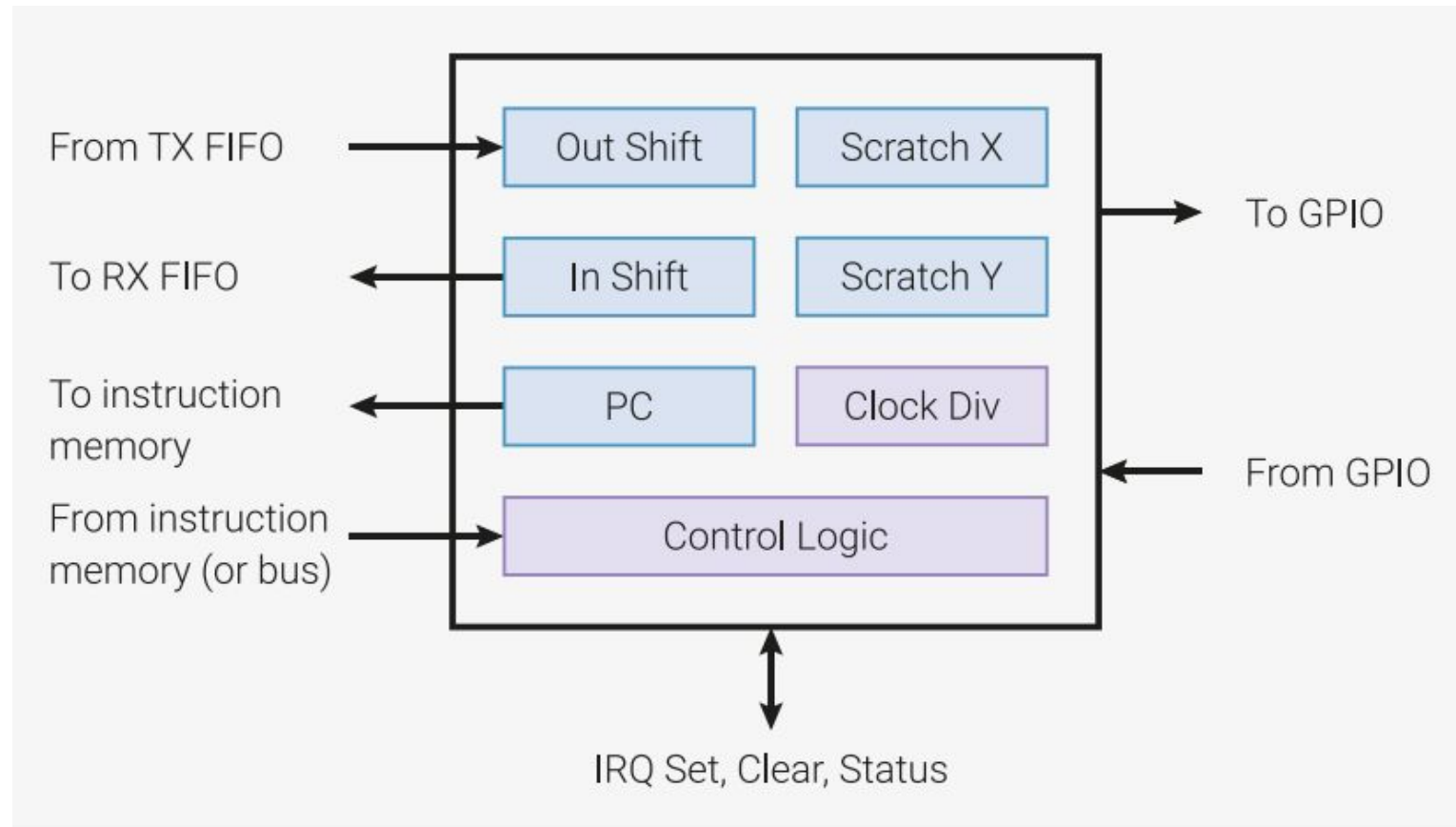
PIO – Can be programmed as:

The programmable input/output block (PIO) is a versatile hardware interface. It can support a variety of IO standards, including:

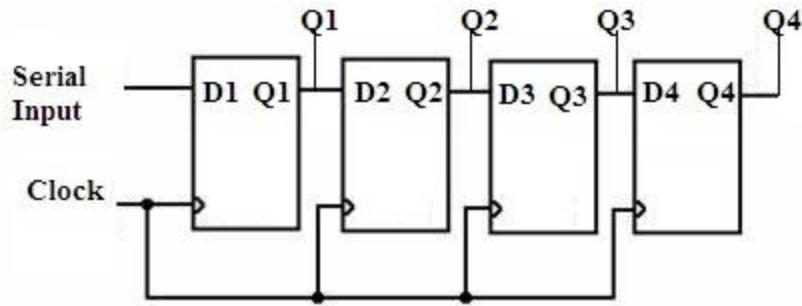


State Machine - Overview

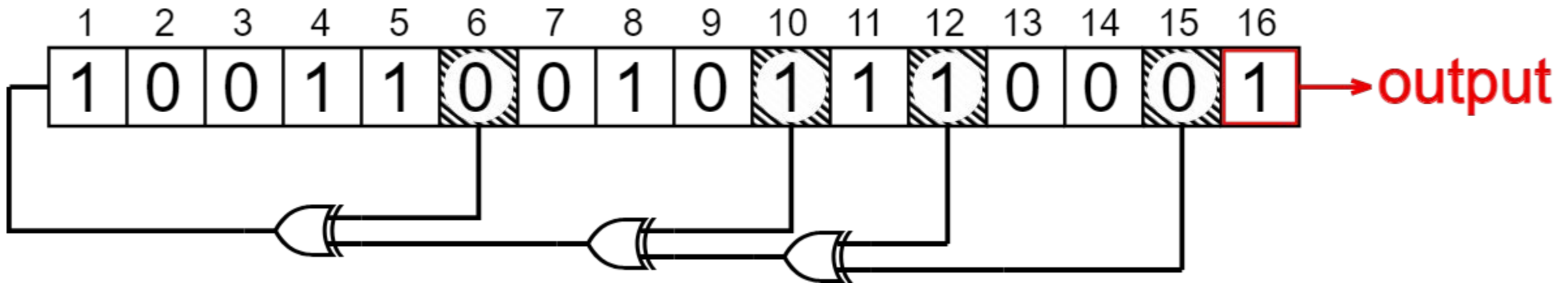
State machines are generally autonomous, and system software interacts through DMA, interrupts and control registers



Shift Registers



Clock Pulse	Q1	Q2	Q3	Q4
0	1	0	0	1
1	1	1	0	0
2	0	1	1	0
3	0	0	1	1

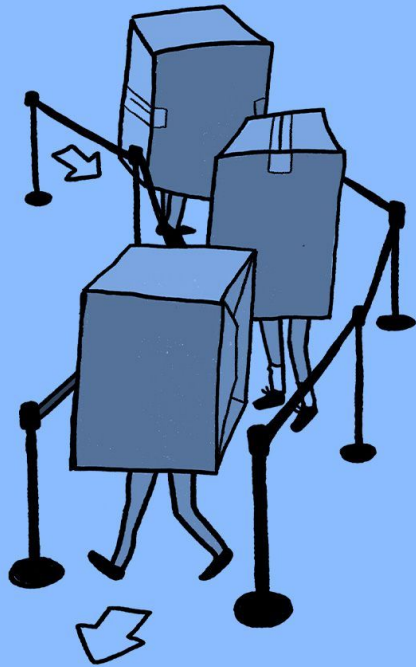


Scratch Registers



*Use as rough
work register for
calculation*

FIFO



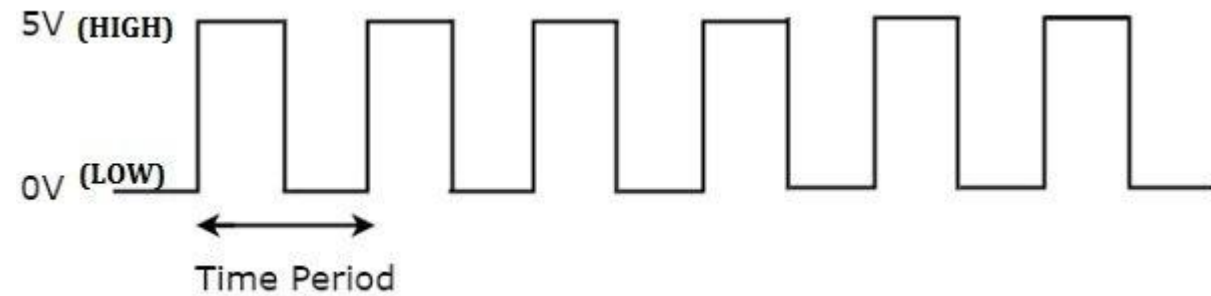
First In, First Out (FIFO)

['fərst 'in 'fərst 'aut]

An asset and inventory management approach in which assets produced or acquired first are sold, used, or disposed of first.

4	3	2	1
8	7	6	5
12	11	10	9
16	15	14	13
20	19	18	17

Clock divider



PIO programming

Programs for common interfaces, such as UART, SPI, or I2C, are available in the PIO library, so in many cases, it is not necessary to write PIO programs. However, the PIO is much more flexible when programmed directly, supporting a wide variety of interfaces which may not have been foreseen by its designers.

JMP, WAIT, IN, OUT, PUSH, PULL, MOV, IRQ, and SET

```
.program blink

set pindirs, 1

loop:
    set pins, 1 [31]
    set pins, 0 [31]
    jmp loop
```

ILO

Illustrate the PIO instructions and the control flow

Control Flow

- On every system clock cycle, each state machine *fetches, decodes and executes* one instruction.
- Each instruction takes *precisely one cycle*, unless it explicitly stalls (such as the WAIT instruction).
- Instructions may also *insert a delay of up to 31 cycles* before the next instruction is executed to aid the writing of cycle-exact programs.
- The *program counter, or PC*, points to the location in the instruction memory being executed on this cycle. Generally, PC increments by one each cycle, wrapping at the end of the instruction memory.
- *Jump instructions are an exception* and explicitly provide the next value that PC will take.

fetches, decodes and executes

precisely one cycle

delay of up to 31 cycles

Jump instructions are an exception

program counter, or PC

Control Flow

Our example assembly program (listed as .program squarewave above) shows both of these concepts in practice. It drives a 50/50 duty cycle square wave onto a GPIO, with a period of four cycles. Using some other features (e.g. side-set) this can be made as low as two cycles.

```
7 .program squarewave
8     set pindirs, 1    ; Set pin to output
9 again:
10     set pins, 1 [1]   ; Drive pin high and then delay for one cycle
11     set pins, 0       ; Drive pin low
12     jmp again         ; Set PC to label `again`
```

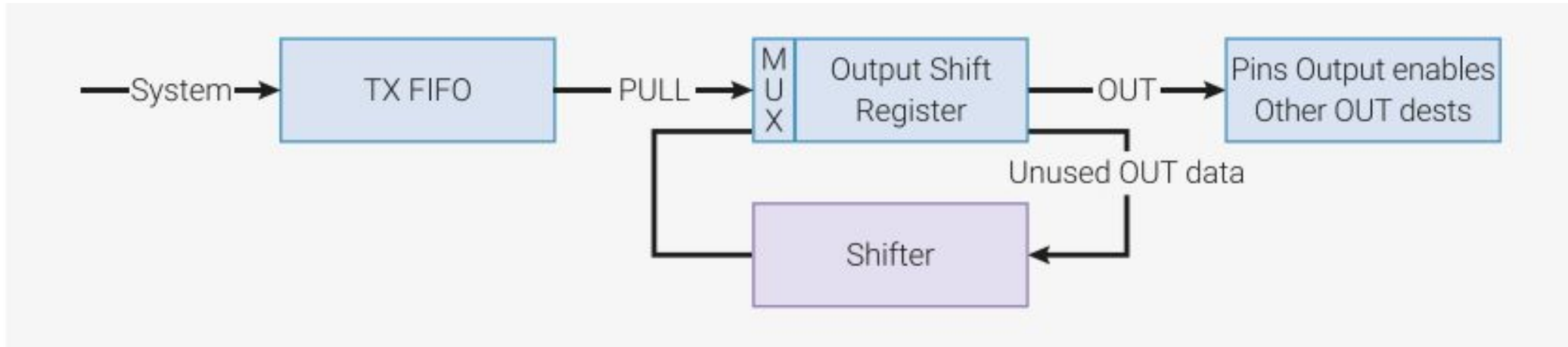
Registers

- Output Shift Register (OSR)
- Input Shift Register (ISR)
- Shift Counters
- Scratch Registers
- FIFOs

ILO

Show the need for different registers in
PIO

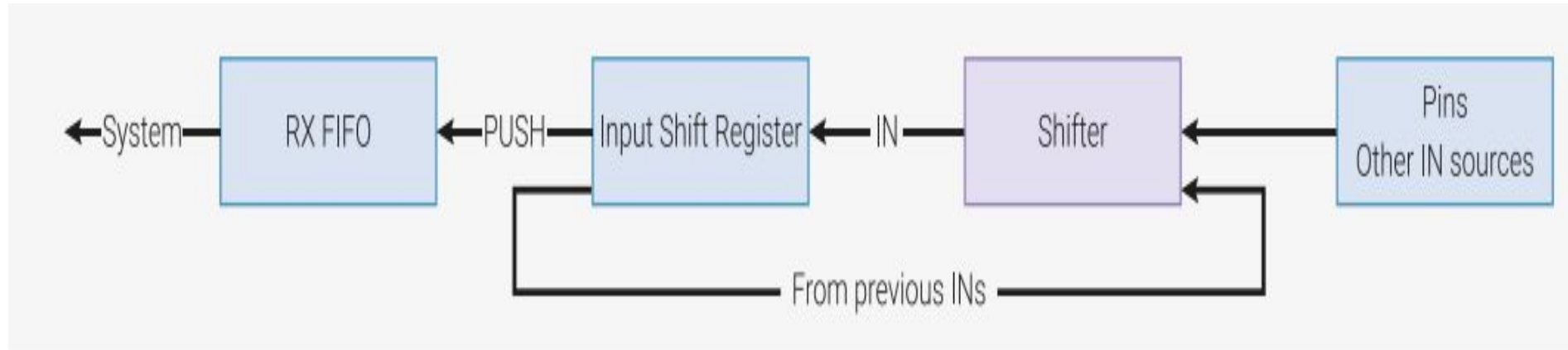
Output Shift Register (OSR)



The Output Shift Register (OSR) holds and shifts output data, between the TX FIFO and the pins (or other destinations, such as the scratch registers).

- PULL instructions:
- OUT instructions
- The OSR -> zeroes as data is shifted out
- The state machine will automatically refill the OSR from the FIFO on an OUT instruction (Autopull)
- Shift direction can be left/right, configurable by the processor via configuration registers

Input Shift Register (ISR)



- IN instructions shift 1...32 bits at a time into the register
- PUSH instructions write the ISR contents to the RX FIFO.
- The ISR is cleared to all-zeroes when pushed.
- The state machine will automatically push the ISR on an IN instruction,(Autopull)
- Shift direction can be left/right, configurable by the processor via configuration registers

Shift Counters

- Counter – counts the shifts
 - Input shift counter – inputs
 - Output shift counter – Outputs
-
- A successful
 - PULL – Output shift counter – 0
 - PUSH – Input shift counter - 0

Write

Flip Flop 1	Flip Flop 2	Flip Flop 3	Flip Flop 4

1 0 0 1
Data Input

Q

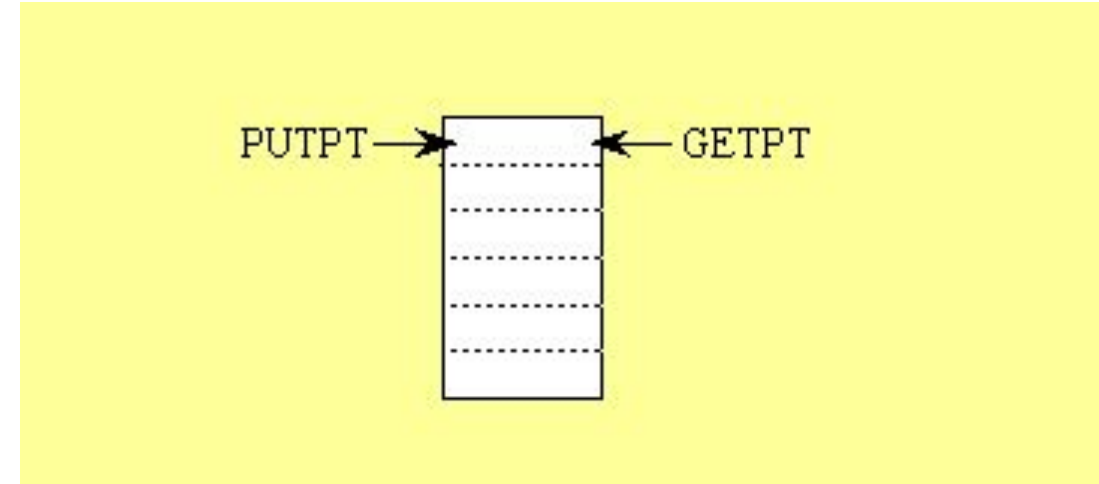
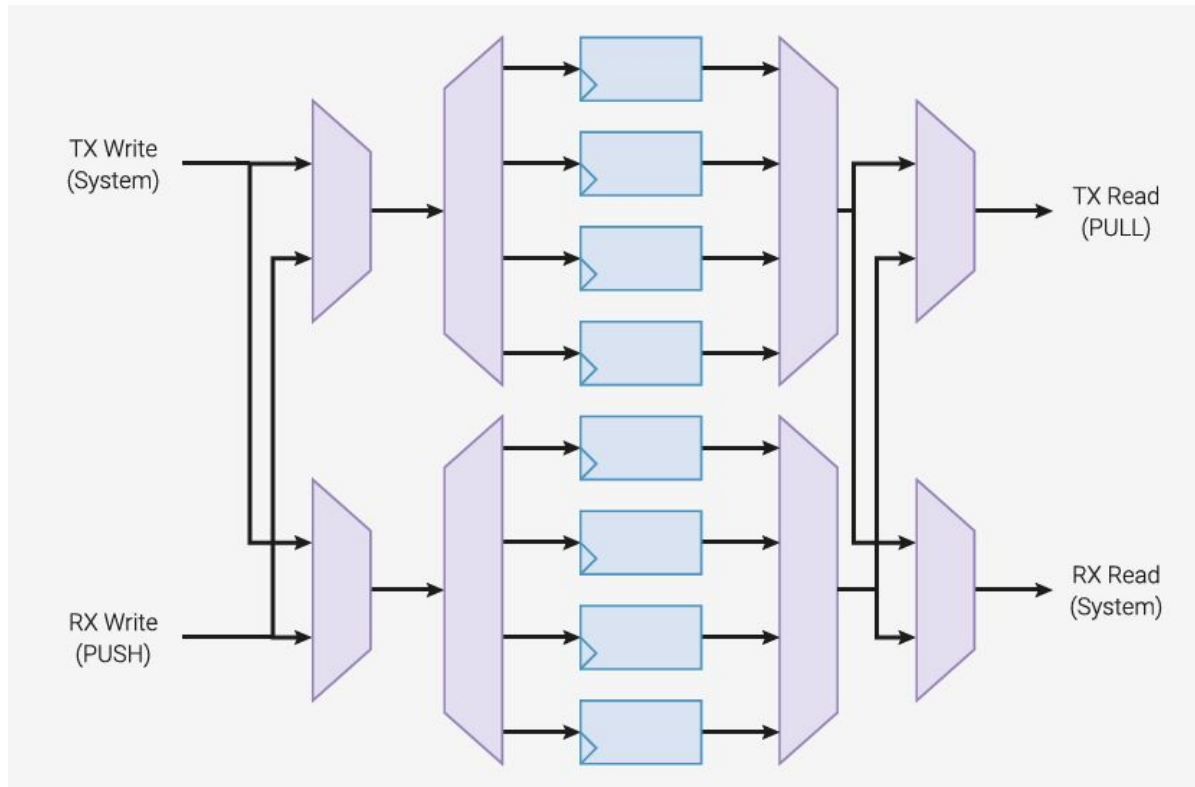


Scratch Registers

- Each state machine has two 32-bit internal scratch registers, called X and Y.
- They are used as:
 - Source/destination for IN/OUT/SET/MOV
 - Source for branch conditions

```
1 .program ws2812_led
2
3 public entry_point:
4     pull
5     set x, 23      ; Loop over 24 bits
6 bitloop:
7     set pins, 1    ; Drive pin high
8     out y, 1 [5]   ; Shift 1 bit out, and write it to y
9     jmp !y skip    ; Skip the extra delay if the bit was 0
10    nop [5]
11 skip:
12    set pins, 0 [5]
13    jmp x-- bitloop ; Jump if x nonzero, and decrement x
14    jmp entry_point
```


- **SHIFTCTRL_FJOIN Option**
- **Data Request (DREQ) Signals**
- **Decoupling Timing:**
- **FIFOs for Data Transfer**

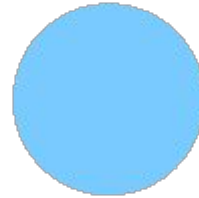


FIFO

Stalling

State machines may momentarily pause execution for a number of reasons

- A **WAIT** instruction
- A blocking **PULL/ PUSH**
- An **IRQ WAIT** instruction
- An **OUT** instruction when autopull is enabled
- An **IN** instruction when autopush is enabled



Pin Mapping

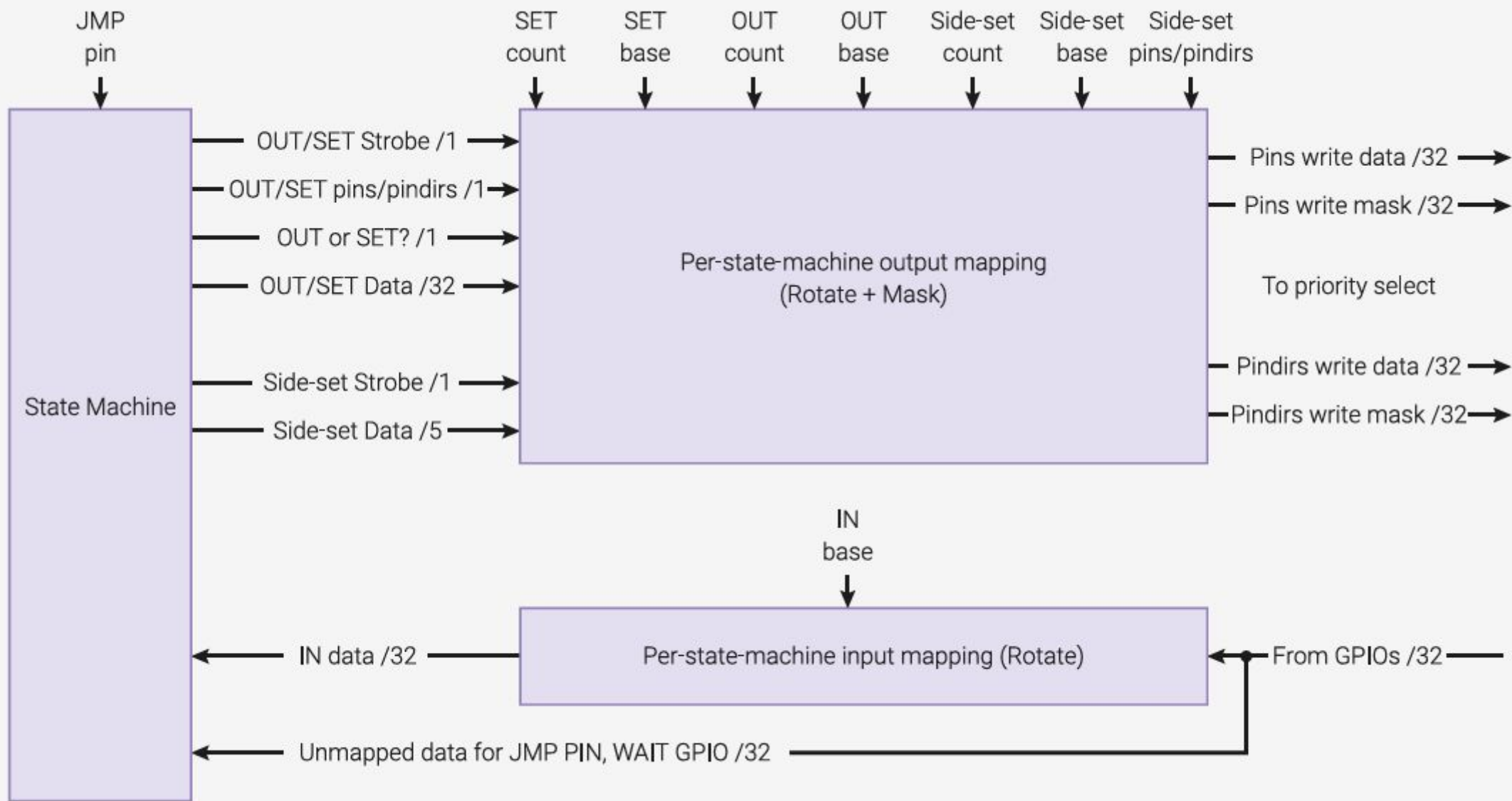
- PIO controls the output level and direction of up to 32 GPIOs,
- and can observe their input levels.

PINCTRL register

4

operations

- Out
- In
- Set
- Side set



IRQ FLAGS

- RQ flags are state bits which can be set or cleared by state machines or the system.

2

Uses

- Asserting system level interrupts
- Synchronising execution between two state machines

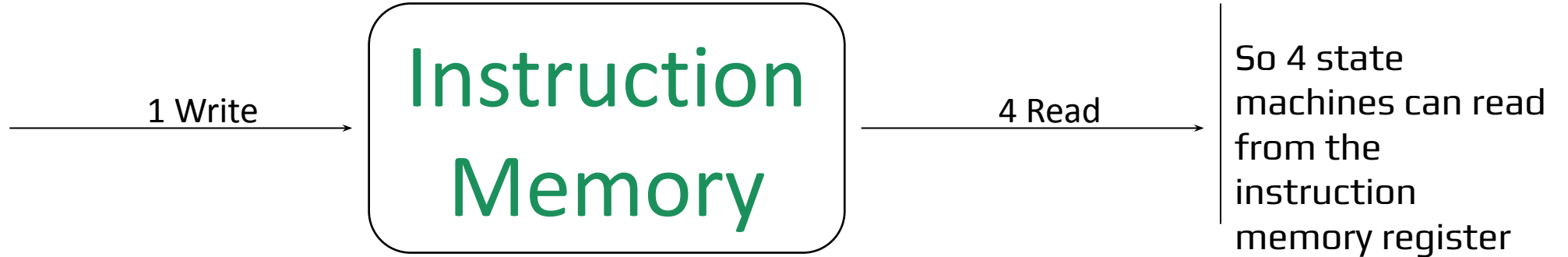
Summarize IRQ flags and interaction between state machines

Control registers

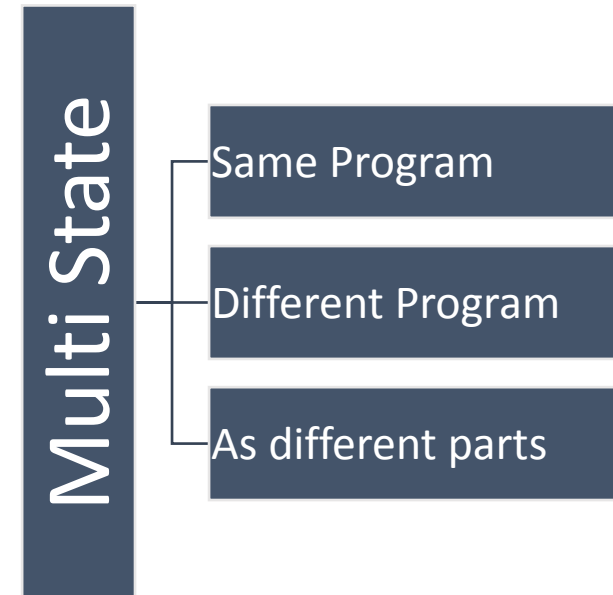
IRQ0_INTE

IRQ1_INTE

Interactions Between State Machines



3 ways to apply the
multiple **state**
machines



PIO Assembler

- Directives
- Expressions
- Comments
- Instructions
- Pseudo instructions

ILO

Illustrate the instruction set with
example



PIO Assembler

- Directives
- Expressions
- Comments
- Instructions
- Pseudo instructions

ILO

Illustrate the instruction set with
example

PIO Assembler



- Directives

directives are special commands or instructions used to configure and control the behavior of state machines within the PIO subsystem. These directives are not executed by the state machine like normal instructions but rather guide the state machine's operation and define its configuration

```
11 .program squarewave_wrap
12 ; Like squarewave, but use the state machine's .wrap hardware instead of an
13 ; explicit jmp. This is a free (0-cycle) unconditional jump.
14
15     set pindirs, 1    ; Set pin to output
16 .wrap_target
17     set pins, 1 [1]   ; Drive pin high and then delay for one cycle
18     set pins, 0 [1]   ; Drive pin low and then delay for one cycle
19 .wrap
```

PIO Assembler



- Expressions

<code><expression> + <expression></code>	The sum of two expressions
<code><expression> - <expression></code>	The difference of two expressions
<code><expression> * <expression></code>	The multiplication of two expressions
<code><expression> / <expression></code>	The integer division of two expressions
<code>- <expression></code>	The negation of another expression
<code>:: <expression></code>	The bit reverse of another expression

PIO Assembler



- Comments

Line comments are supported with `//` or `;` C-style
block comments are supported via `/*` and `*/`

```
11 .program squarewave_wrap
12 ; Like squarewave, but use the state machine's .wrap hardware instead of an
13 ; explicit jmp. This is a free (0-cycle) unconditional jump.
14
15     set pindirs, 1 ; Set pin to output
16 .wrap_target
17     set pins, 1 [1] ; Drive pin high and then delay for one cycle
18     set pins, 0 [1] ; Drive pin low and then delay for one cycle
19 .wrap
```



Instruction *<instruction> (side <side_set_value>) ([<delay_value>])*

```
11 .program squarewave_wrap
12 ; Like squarewave, but use the state machine's .wrap hardware instead of an
13 ; explicit jmp. This is a free (0-cycle) unconditional jump.
14
15     set pindirs, 1    ; Set pin to output
16 .wrap_target
17     set pins, 1 [1]   ; Drive pin high and then delay for one cycle
18     set pins, 0 [1]   ; Drive pin low and then delay for one cycle
19 .wrap
```



Instruction *<instruction> (side <side_set_value>) ([<delay_value>])*

Bit:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JMP	0	0	0	Delay/side-set					Condition			Address				
WAIT	0	0	1	Delay/side-set					Pol	Source		Index				
IN	0	1	0	Delay/side-set					Source			Bit count				
OUT	0	1	1	Delay/side-set					Destination			Bit count				
PUSH	1	0	0	Delay/side-set					0	IfF	Blk	0	0	0	0	0
PULL	1	0	0	Delay/side-set					1	IfE	Blk	0	0	0	0	0
MOV	1	0	1	Delay/side-set					Destination			Op		Source		
IRQ	1	1	0	Delay/side-set					0	Clr	Wait	Index				
SET	1	1	1	Delay/side-set					Destination			Data				



Pseudoinstructions

Currently pioasm provides **one** pseudoinstruction, as a convenience:

nop Assembles to **mov y, y**. "No operation", has no particular side effect, but a useful vehicle for a side-set operation or an extra delay

Functional Details

- Side-set
- Program Wrapping
- FIFO Joining
- Autopush and Autopull
- Clock Dividers
- GPIO Mapping
- Forced and EXEC'd Instructions

ILO

Describe the functional details of state machines

Functional Details

Side-set

REMEMBER

Side-set is a feature that allows state machines to change the level or direction of up to 5 pins, concurrently with the main execution of the instruction.

```
1 .program spi_tx_fast
2 .side_set 1
3
4 loop:
5     out pins, 1    side 0
6     jmp loop      side 1
```


Functional Details

Program Wrapping

REMEMBER

PIO programs often have an "outer loop": they perform the same sequence of steps, repetitively, as they transfer a stream of data between the FIFOs and the outside world. The square wave program from the introduction is a minimal example of this:

```
7 .program squarewave
8     set pindirs, 1    ; Set pin to output
9 again:
10     set pins, 1 [1]   ; Drive pin high and then delay for one cycle
11     set pins, 0       ; Drive pin low
12     jmp again         ; Set PC to label `again`
```

```
11 .program squarewave_wrap
12 ; Like squarewave, but use the state machine's .wrap hardware instead of an
13 ; explicit jmp. This is a free (0-cycle) unconditional jump.
14
15     set pindirs, 1    ; Set pin to output
16 .wrap_target
17     set pins, 1 [1]   ; Drive pin high and then delay for one cycle
18     set pins, 0 [1]   ; Drive pin low and then delay for one cycle
19 .wrap
```

Functional Details

FIFO Joining

By default, each state machine possesses a 4-entry FIFO in each direction: one for data transfer from system to state machine (TX), the other for the reverse direction (RX). However, many applications do not require bidirectional data transfer between the system and an individual state machine, but may benefit from deeper FIFOs

Functional Details

Autopush and Autopull

With each OUT instruction, the OSR gradually empties, as data is shifted out. Once empty, it must be refilled: for example, a PULL transfers one word of data from the TX FIFO to the OSR. Similarly, the ISR must be emptied once full. One approach to this is a loop which performs a PULL after an appropriate amount of data has been shifted this creates a delay in the system, hence to overcome this autopush and autopull is used.

Functional Details

Clock Dividers

PIO runs off the system clock, but this is simply too fast for many interfaces, and the number of Delay cycles which can be inserted is limited. Some devices, such as UART, require the signalling rate to be precisely controlled and varied, and ideally multiple state machines can be varied independently while running identical programs. Each state machine is equipped with a clock divider, for this purpose

Functional Details

GPIO Mapping

Internally, PIO has a 32-bit register for the output levels of each GPIO it can drive, and another register for the output enables (Hi/Lo-Z). On every system clock cycle, each state machine can write to some or all of the GPIOs in each of these registers.

Priority in output is given to the highest numbered state machine

Functional Details

Forced and EXEC'd
Instructions

Besides the instruction memory, state machines can execute instructions from 3 other sources

- **MOV EXEC** which executes an instruction from some register Source
- **OUT EXEC** which executes data shifted out from the OSR
- The **SMx_INSTR** control registers, to which the system can write instructions for immediate execution

